

I am attending the DL class.

Deep Learning

Lecture: Recurrent neural networks

How to make sense of **sequential data**?

- Recurrent neural networks
- Applications
- Beyond sequences

Many real-world problems require to process a signal with a **sequence** structure.

- Sequence classification:
 - sentiment analysis in text
 - activity/action recognition in videos
 - DNA sequence classification
- Sequence synthesis:
 - text synthesis
 - music synthesis
 - motion synthesis
- Sequence-to-sequence translation:
 - speech recognition
 - text translation
 - part-of-speech tagging

$$\{x_1, x_2, x_3, \dots, x_n\}$$

Given a set $\mathcal{X}$, if $S(\mathcal{X})$ denotes the set of sequences of elements from $\mathcal{X}$,

$$S(\mathcal{X}) = \bigcup_{t=1}^{\infty} \mathcal{X}^t,$$

then we formally define:

$$S(\mathcal{X}) = \{x^1, x^2, \dots\}$$

Sequence classification

$$f : S(\mathcal{X}) \rightarrow \Delta^C$$

Sequence synthesis

$$f : \mathbb{R}^d \rightarrow S(\mathcal{X})$$

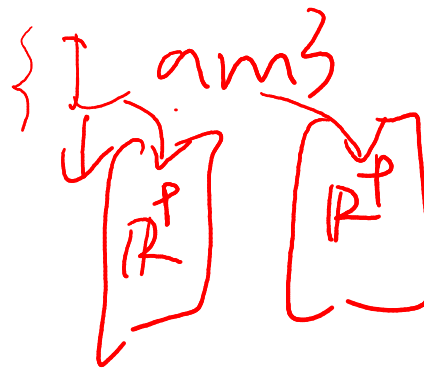
Sequence-to-sequence translation

$$f : S(\mathcal{X}) \rightarrow S(\mathcal{Y})$$

In the rest of the slides, we consider only time-indexed signal, although it generalizes to arbitrary sequences.

$$x^2 = \{\{1 \text{ am}\}, \{am \text{ I}\}\}$$

$$x^1 = \{\{1\}, \{am\}\}$$



Recurrent neural networks

When the input is a sequence $\mathbf{x} \in S(\mathbb{R}^p)$ of variable length $T(\mathbf{x})$, the historical approach is to use a recurrent model which maintains a recurrent state $\mathbf{h}_t \in \mathbb{R}^q$ updated at each time step t .

$\{ \text{I am attending} \dots \}$

$t=0$, p dim vect
 $t=1$, " " $S(X)$
word-to-vec
 $\mathbf{x} = \{ \underline{\mathbb{R}^p}, \underline{\mathbb{R}^p}, \underline{\mathbb{R}^p} \dots \}$

Formally, for $t = 1, \dots, T(\mathbf{x})$,

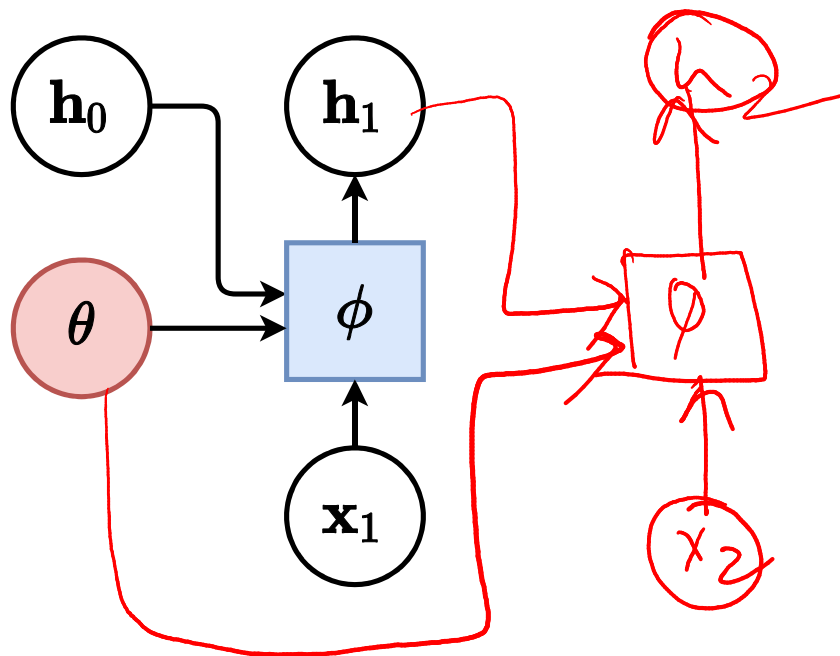
$$\mathbf{h}_t = \phi(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta),$$

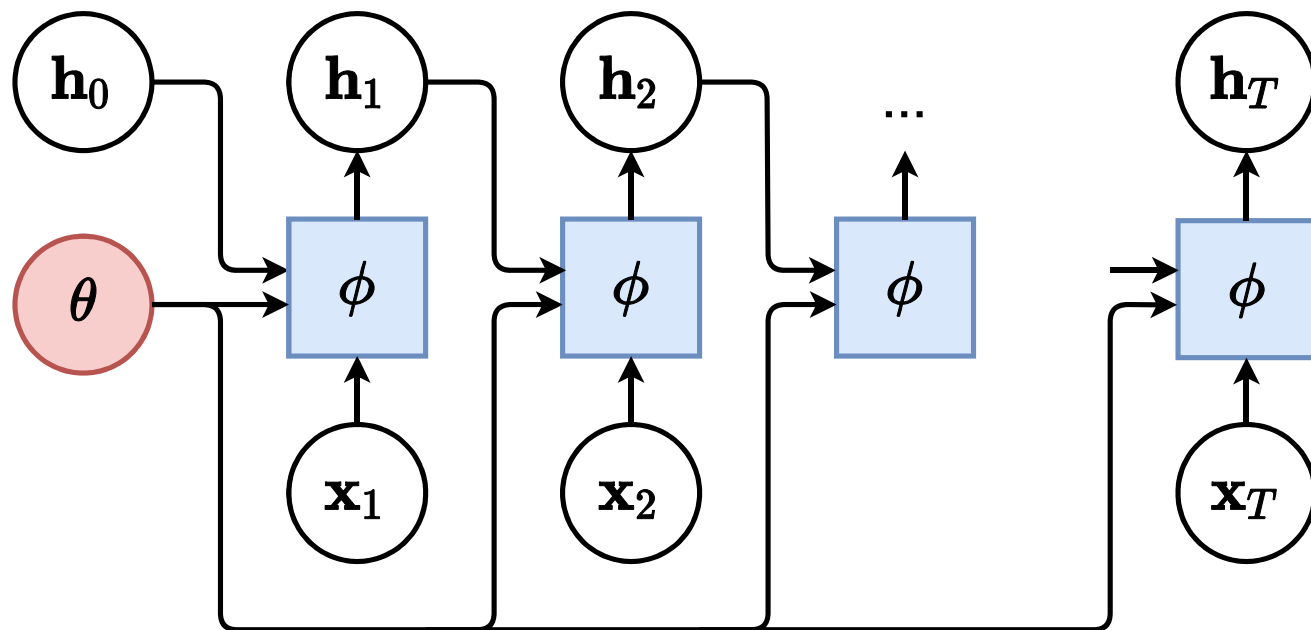
where $\phi : \mathbb{R}^p \times \mathbb{R}^q \rightarrow \mathbb{R}^q$ and $\mathbf{h}_0 \in \mathbb{R}^q$.

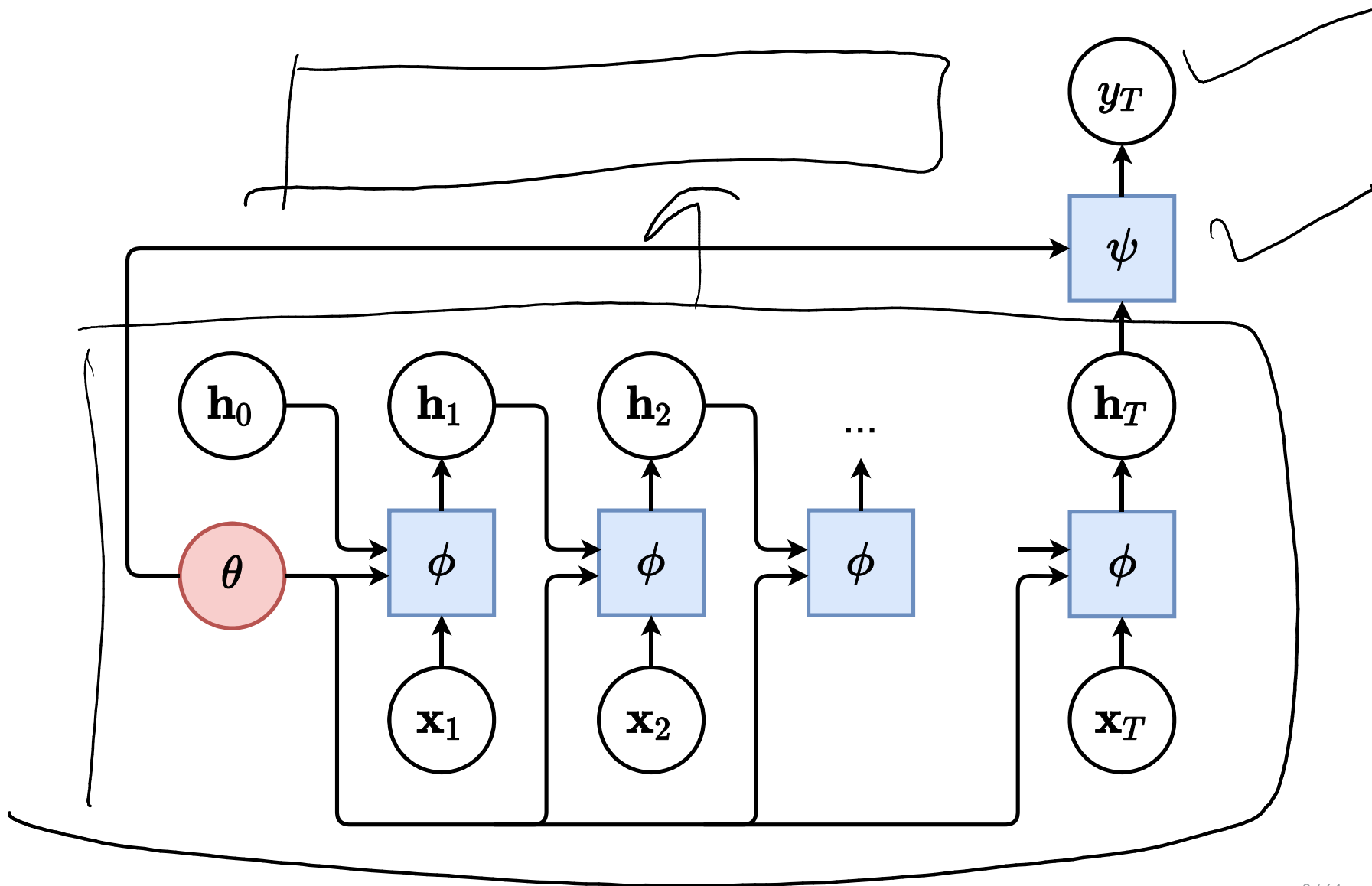
Predictions can be computed at any time step t from the recurrent state,

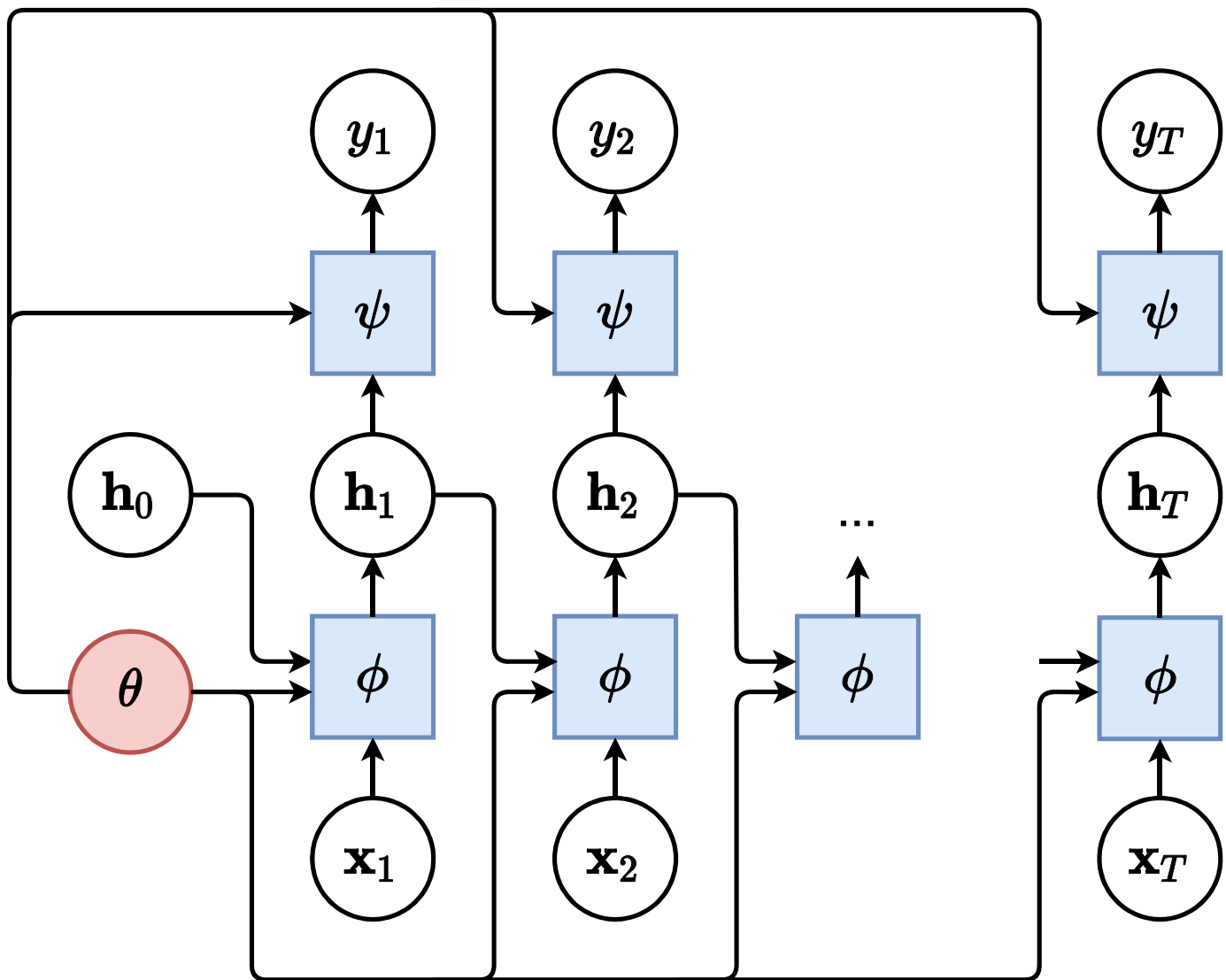
$$y_t = \psi(\mathbf{h}_t; \theta),$$

with $\psi : \mathbb{R}^q \rightarrow \mathbb{R}^C$.









I am hungry

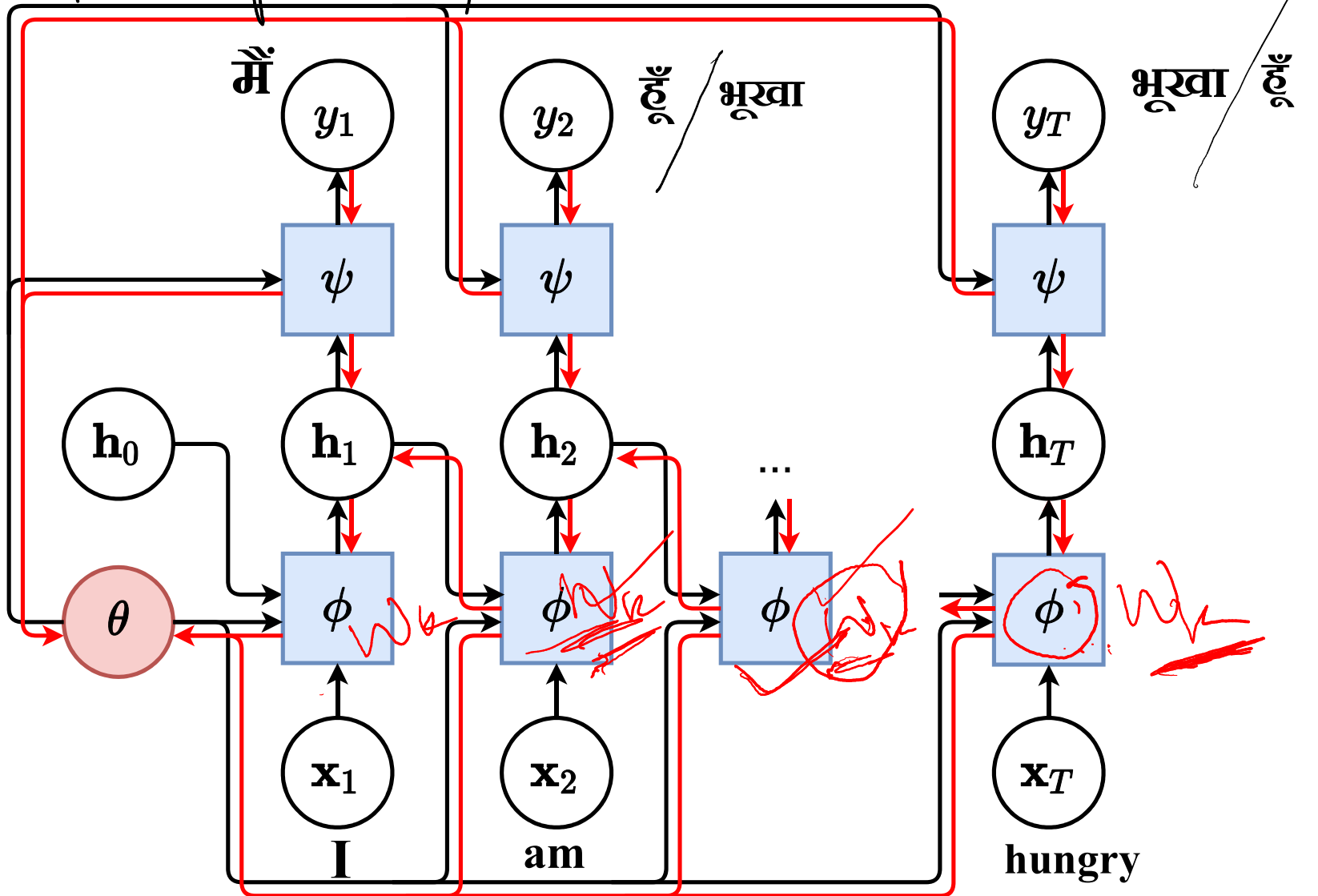


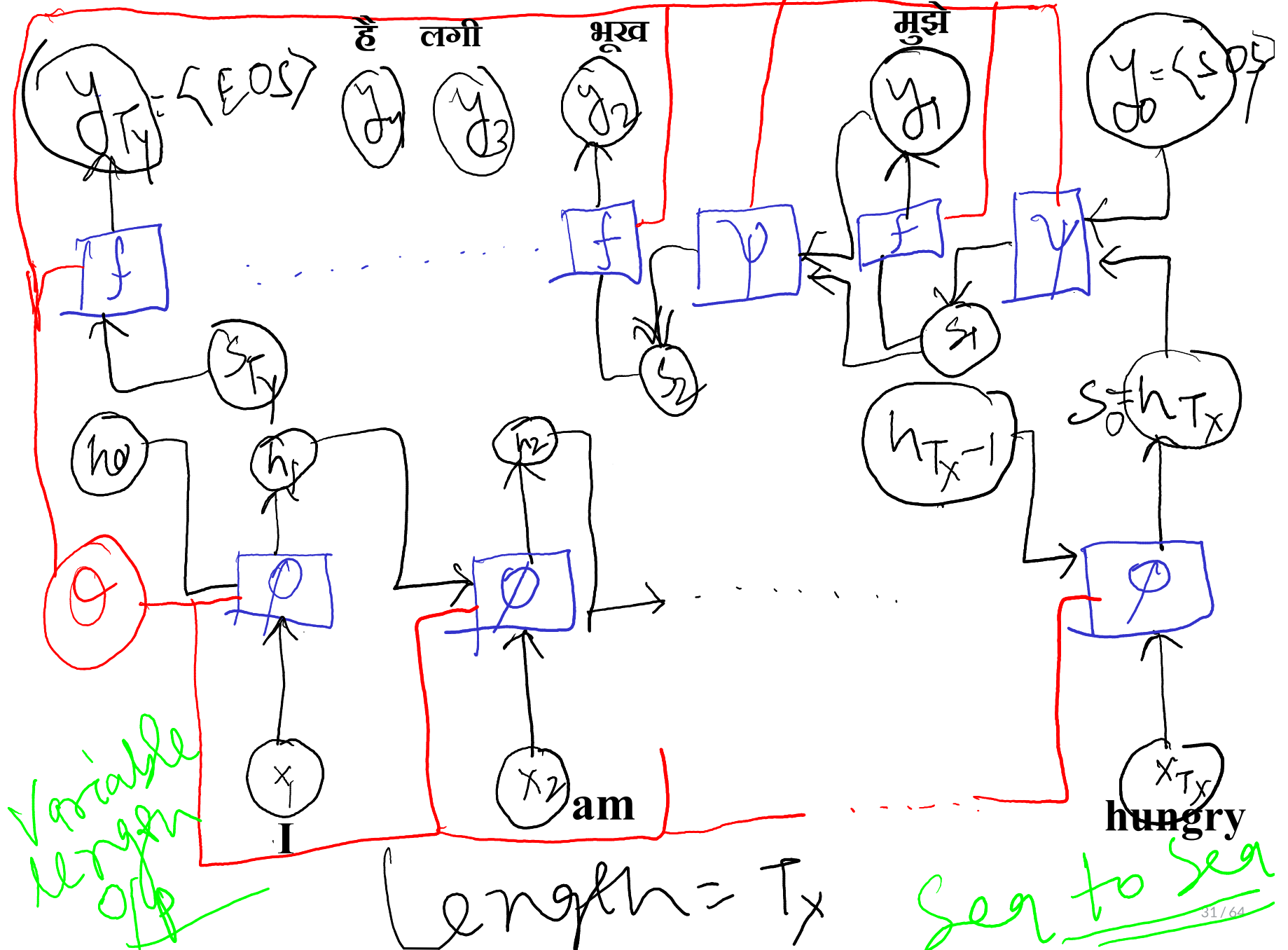
मुझे भूख लगी है

Even though the number of steps T depends on x , this is a standard computational graph, and automatic differentiation can deal with it as usual.

In the case of recurrent neural networks, this is referred to as **backpropagation through time**.

Fixed length ϕ/p



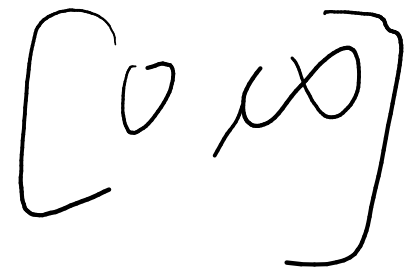


Elman networks

Elman networks consist of ϕ and ψ defined as primitive neuron units, such as logistic regression units

$$\begin{aligned} \mathbf{h}_t &= \sigma_h (\mathbf{W}_{xh}^T \mathbf{x}_t + \mathbf{W}_{hh}^T \mathbf{h}_{t-1} + \mathbf{b}_h) \\ y_t &= \sigma_y (\mathbf{W}_y^T \mathbf{h}_t + b_y) \end{aligned}$$

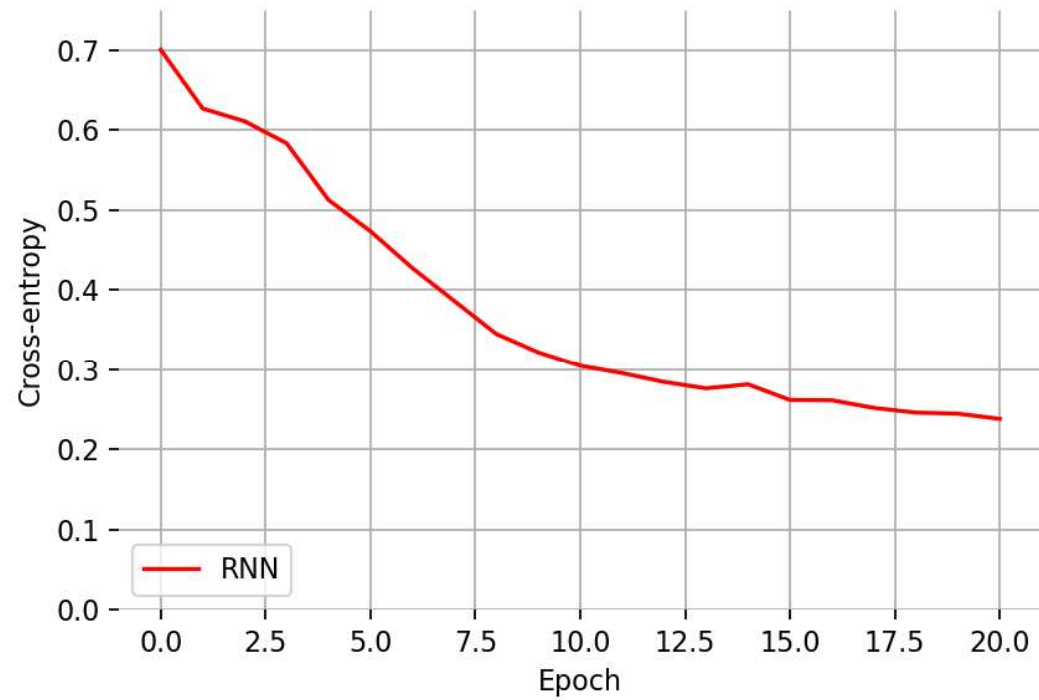
where $\mathbf{W}_{xh} \in \mathbb{R}^{p \times q}$, $\mathbf{W}_{hh} \in \mathbb{R}^{q \times q}$, $\mathbf{b}_h \in \mathbb{R}^q$, $b_y \in \mathbb{R}$, $\mathbf{h}_0 = \mathbf{0}$, and where σ_h and σ_y are non-linear activation functions, such as the sigmoid function, $\tanh$ or ReLU.



Benchmark example

Learn to recognize variable-length sequences that are palindromes. For training, we use sequences of random sizes, from 1 to 10.

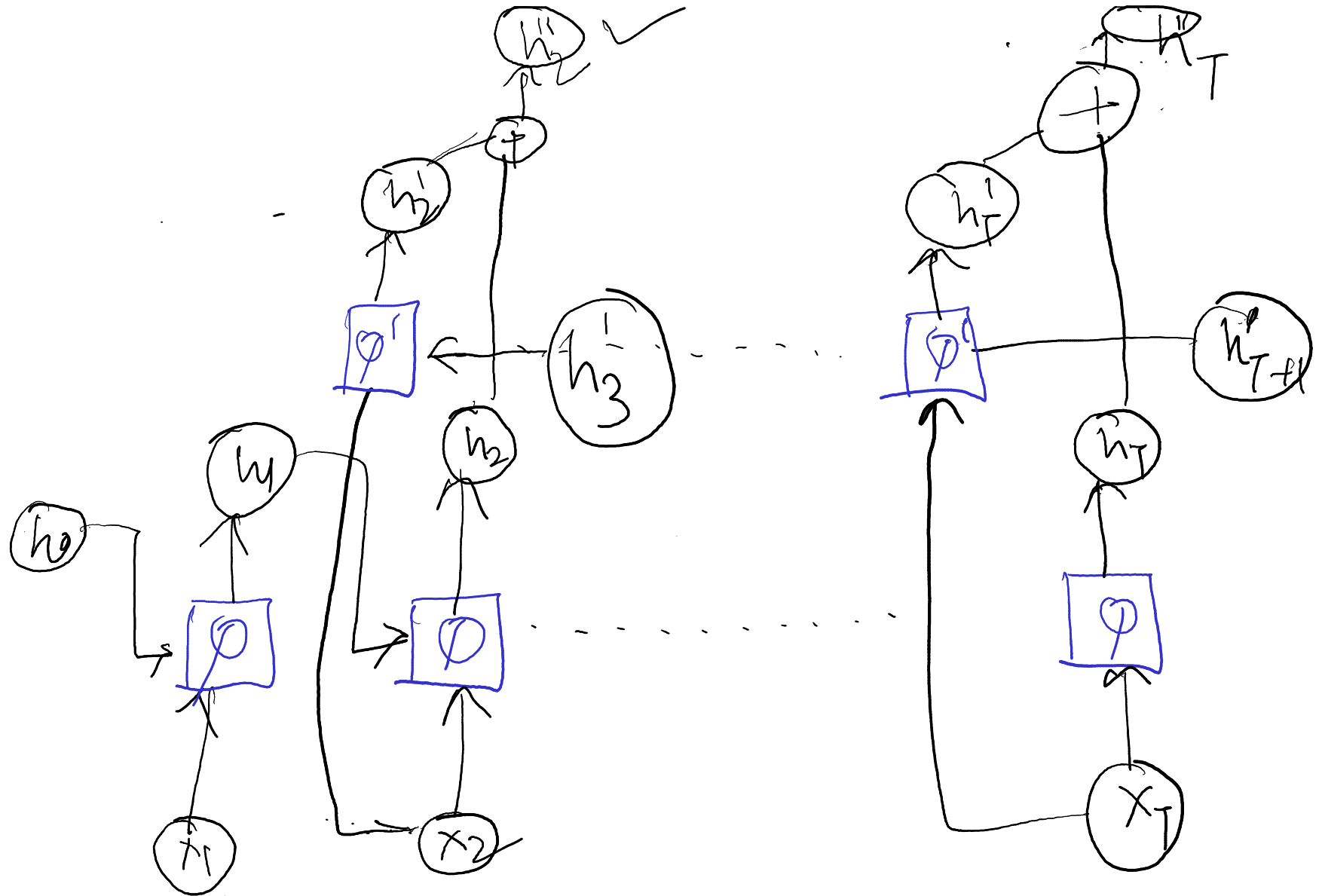
$\mathbf{x}$	y
(1, 2, 3, 2, 1)	1
(2, 1, 2)	1
(3, 4, 1, 2)	0
(0)	1
(1, 4)	0

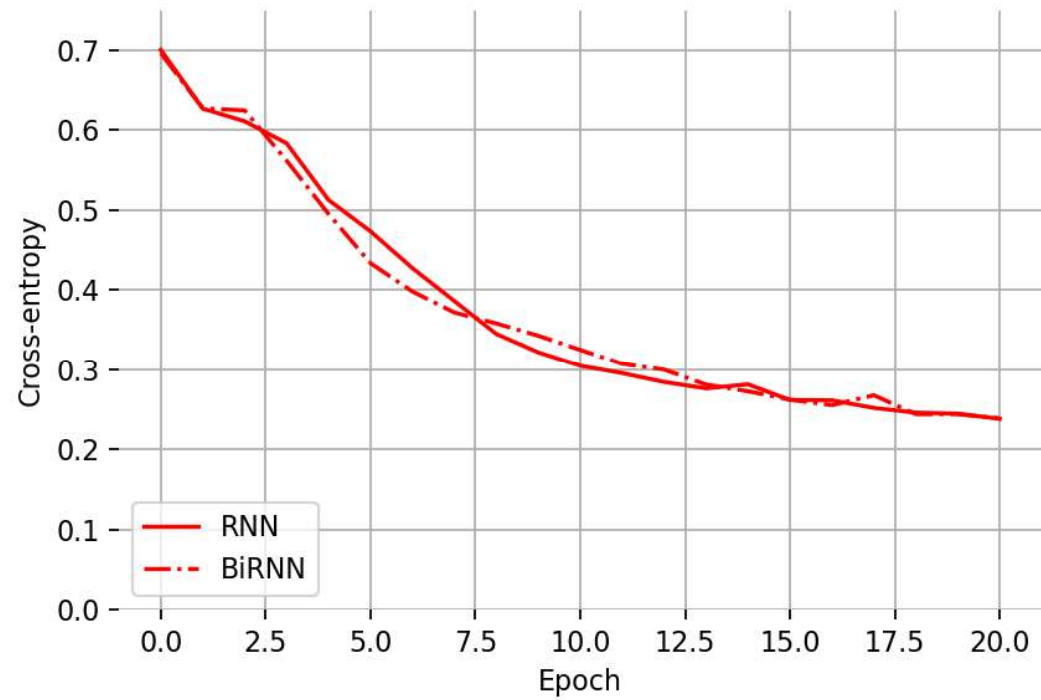


Bidirectional RNNs

Computing the recurrent states forward in time does not make use of future input values $\mathbf{x}_{t+1:T}$, even though there are known.

- RNNs can be made **bidirectional** by consuming the sequence in both directions.
- Effectively, this amounts to run the same (single direction) RNN twice:
 - once over the original sequence $\mathbf{x}_{1:T}$,
 - once over the reversed sequence $\mathbf{x}_{T:1}$.
- The resulting recurrent states of the bidirectional RNN is the concatenation of two resulting sequences of recurrent states.

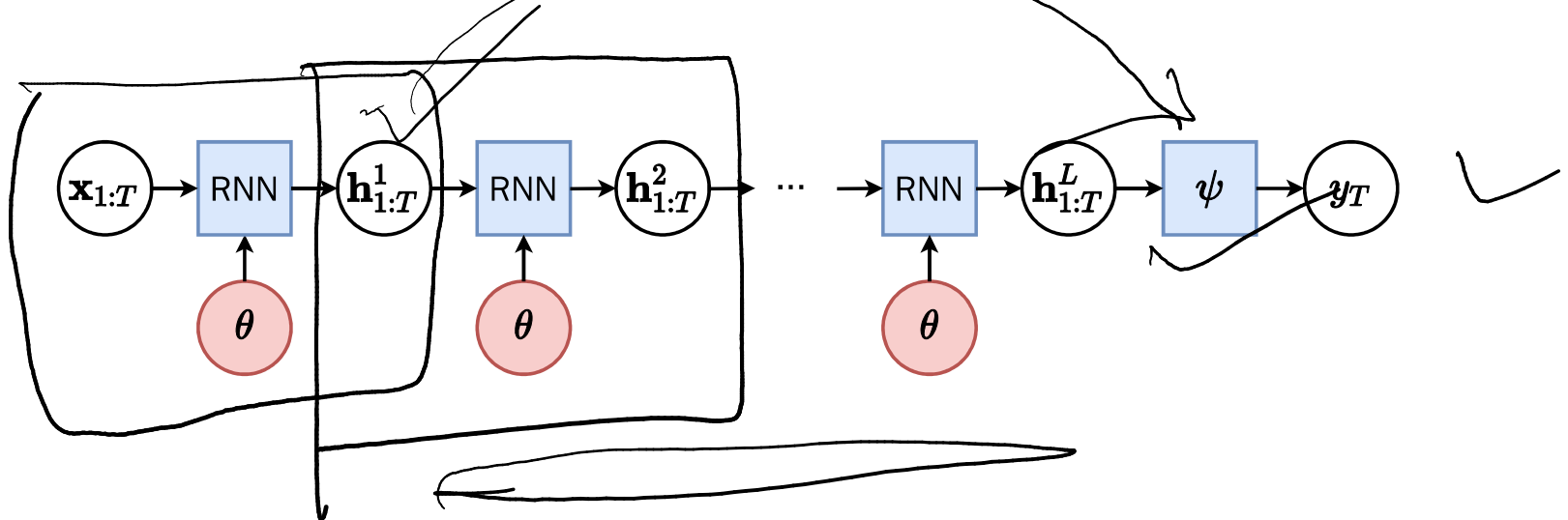


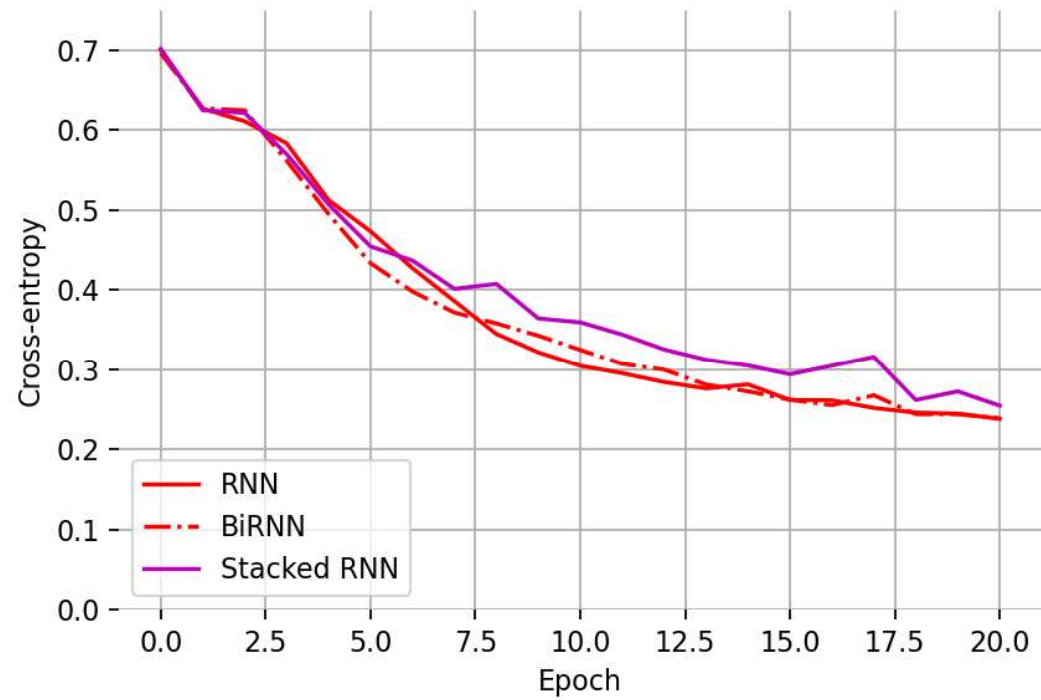


Stacked RNNs

Recurrent networks can be viewed as layers producing sequences $\mathbf{h}_{1:T}^\ell$ of activations.

As for dense layers, recurrent layers can be composed in series to form a **stack** of recurrent networks.

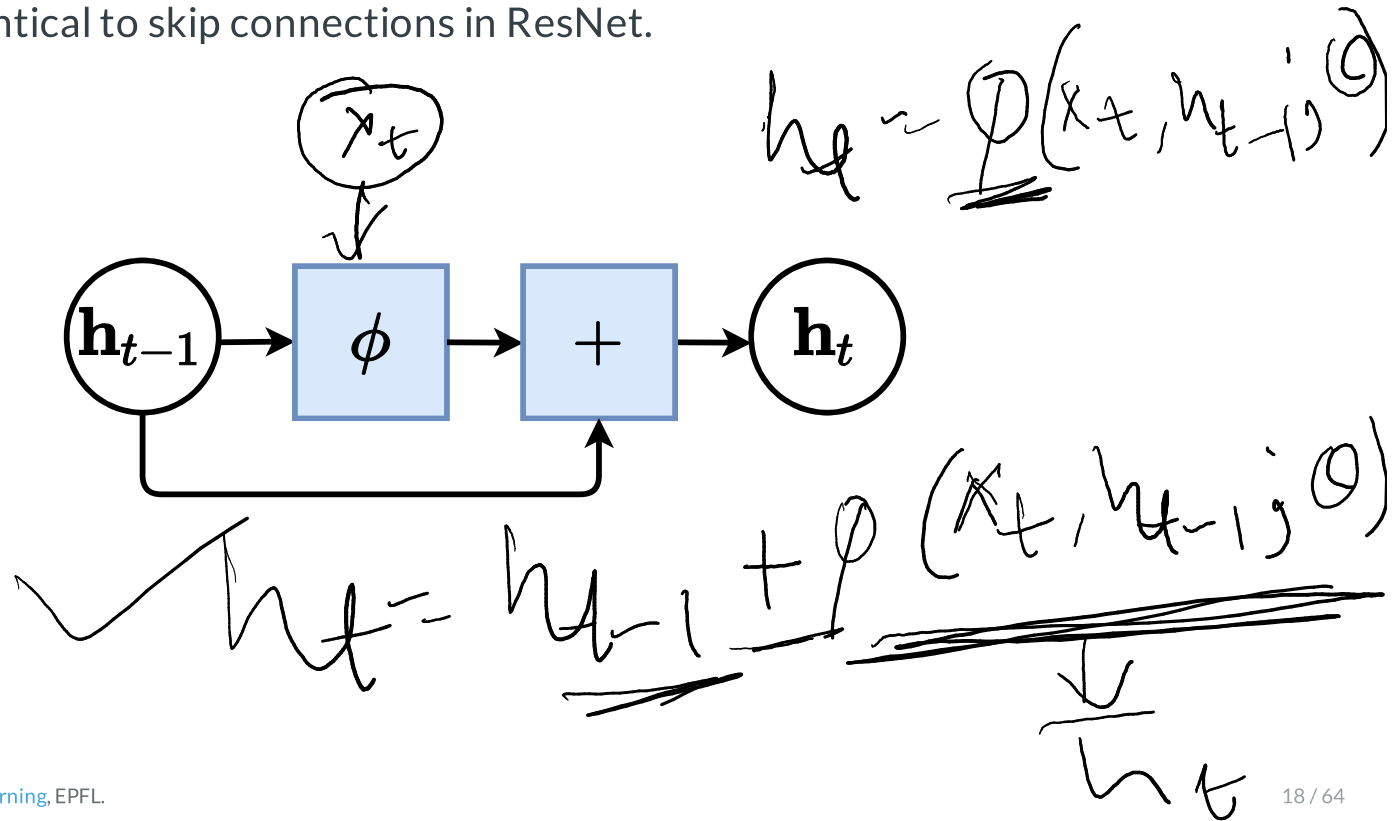




Gating

When unfolded through time, the graph of computation of a recurrent network can grow very deep, and training involves dealing with **vanishing gradients**.

- RNN cells should include a **pass-through**, or additive paths, so that the recurrent state does not go repeatedly through a squashing non-linearity.
- This is identical to skip connections in ResNet.



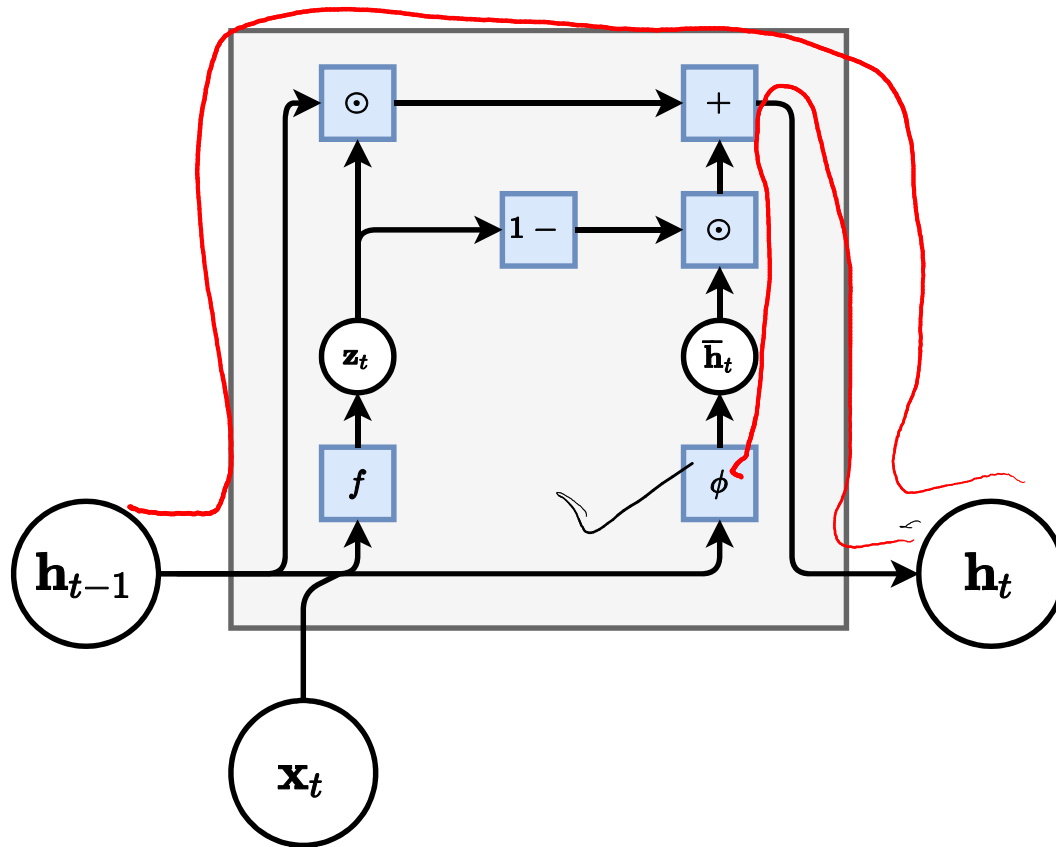
For instance, the recurrent state update can be a per-component weighted average of its previous value $\mathbf{h}_{t-1}$ and a full update $\bar{\mathbf{h}}_t$, with the weighting $\mathbf{z}_t$ depending on the input and the recurrent state, hence acting as a **forget gate**.

Formally,

$$\bar{\mathbf{h}}_t = \phi(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta)$$

$$\mathbf{z}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta)$$

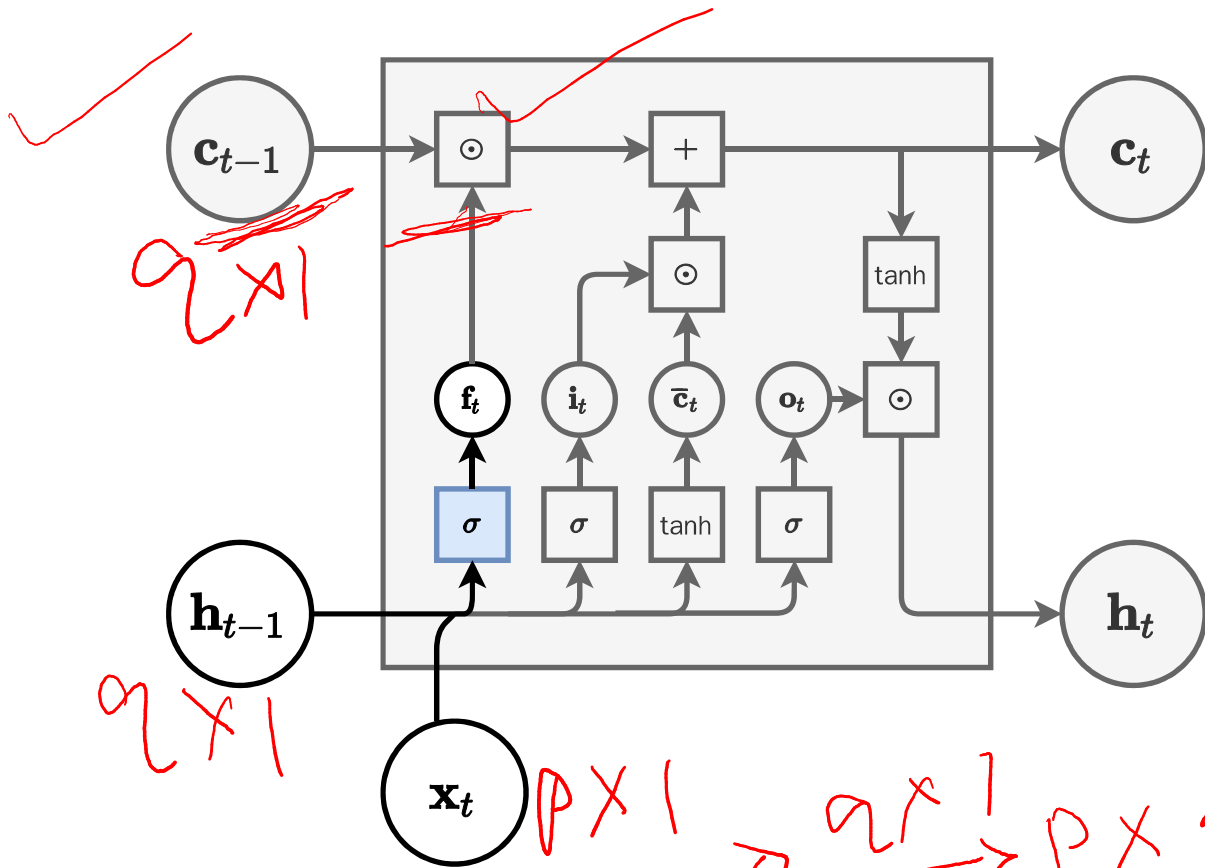
$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \bar{\mathbf{h}}_t.$$



LSTM

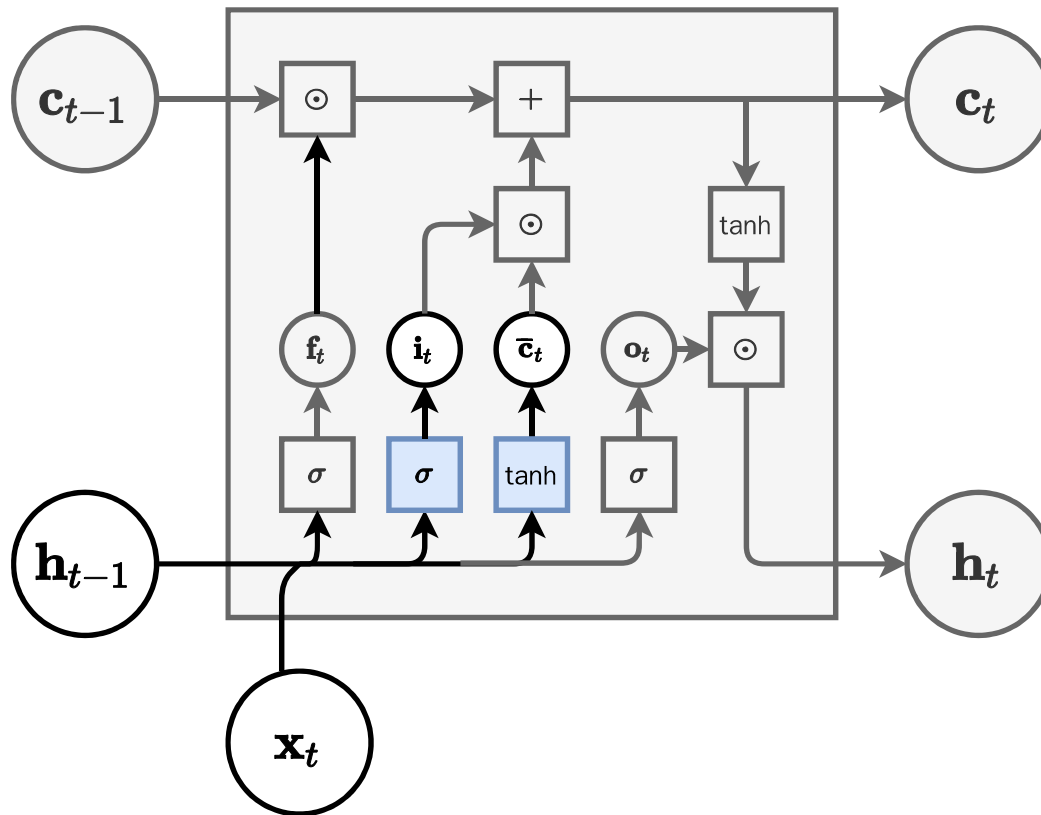
The long short-term memory model (LSTM; Hochreiter and Schmidhuber, 1997) is an instance of the previous gated recurrent cell, with the following changes:

- The recurrent state is split into two parts $\mathbf{c}_t$ and $\mathbf{h}_t$, where
 - $\mathbf{c}_t$ is the cell state and
 - $\mathbf{h}_t$ is output state.
- A forget gate $\mathbf{f}_t$ selects the ~~cell state~~ information to erase.
- An input gate $\mathbf{i}_t$ selects the cell state information to update.
- An output gate $\mathbf{o}_t$ selects the cell state information to output.



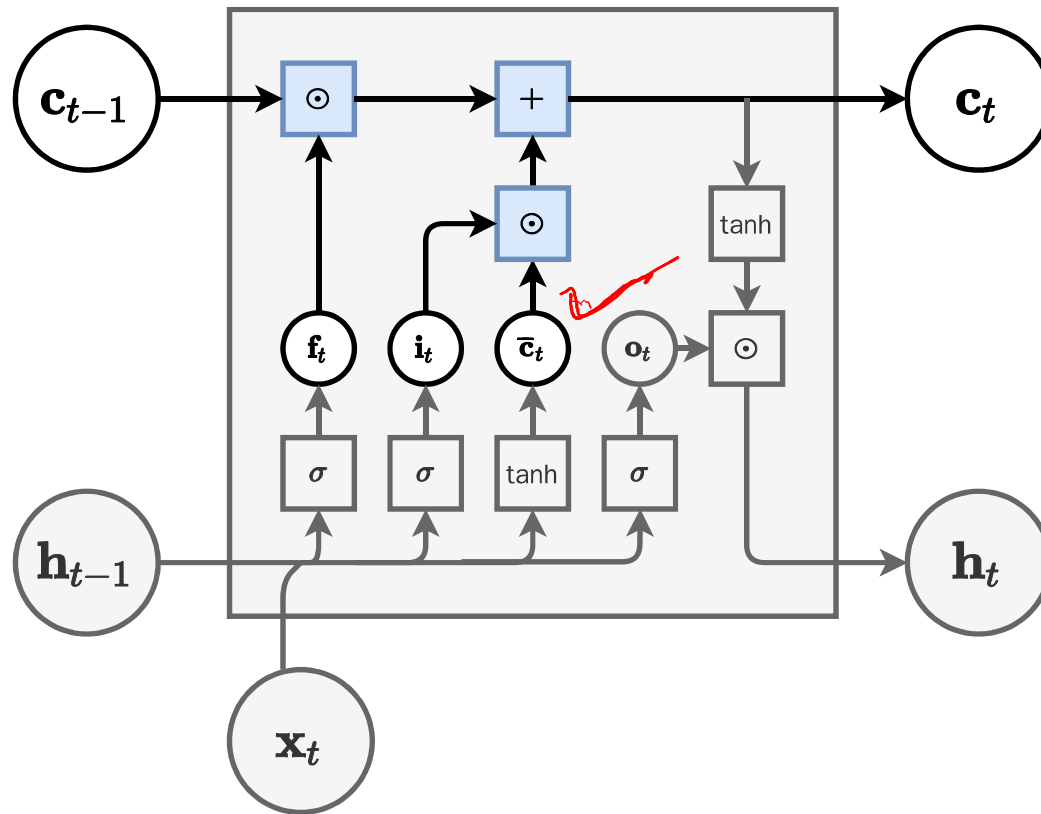
$$f_t = \sigma(\mathbf{W}_f^T [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

$p \times 1$ $\rightarrow$ $q \times 1$ $\rightarrow$ $p \times 1$
 $q \times (p+q) \rightarrow (p+q) \times 1$

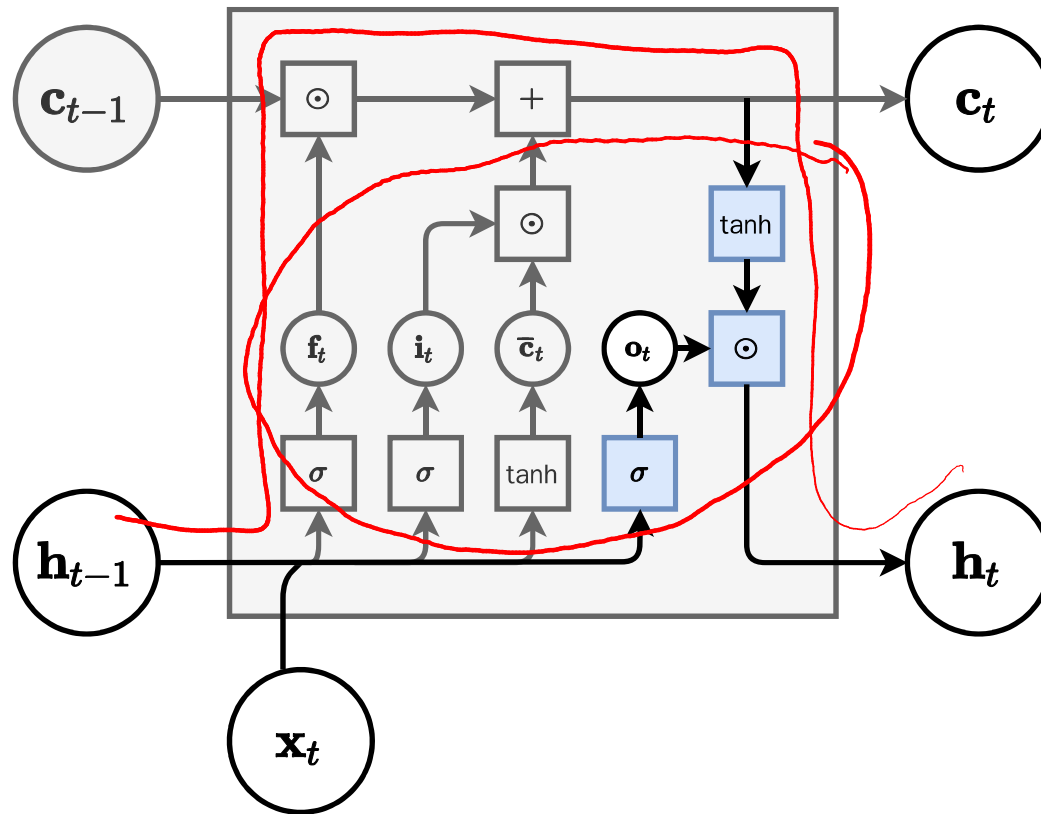


$$i_t = \sigma(W_i^T[h_{t-1}, x_t] + b_i)$$

$$\bar{c}_t = \tanh(W_c^T[h_{t-1}, x_t] + b_c)$$

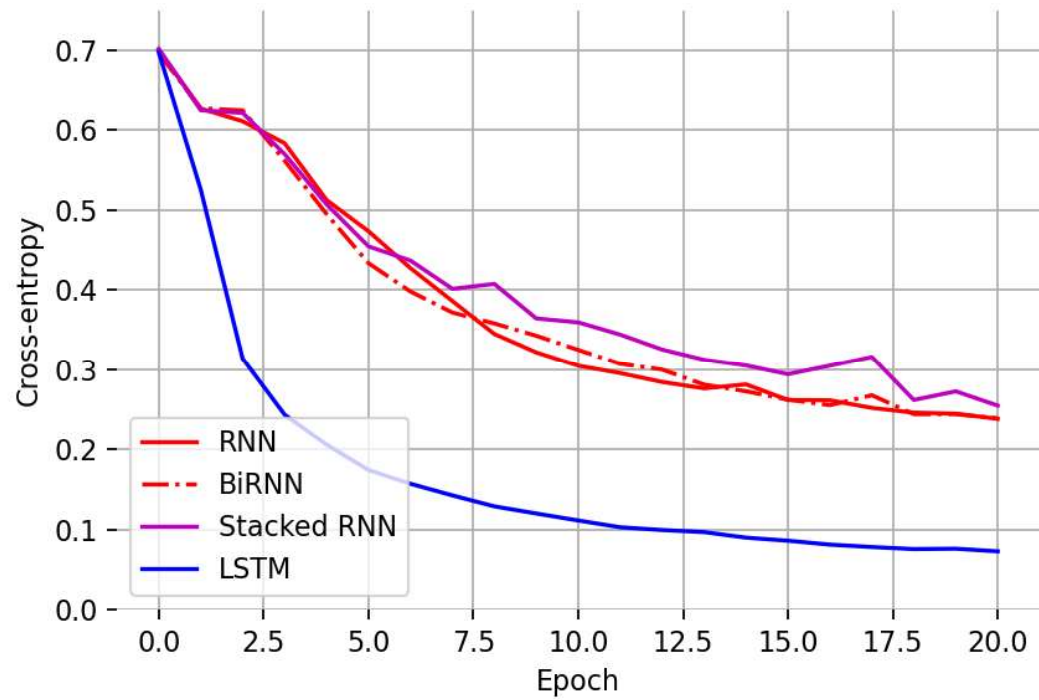


$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \bar{\mathbf{c}}_t$$



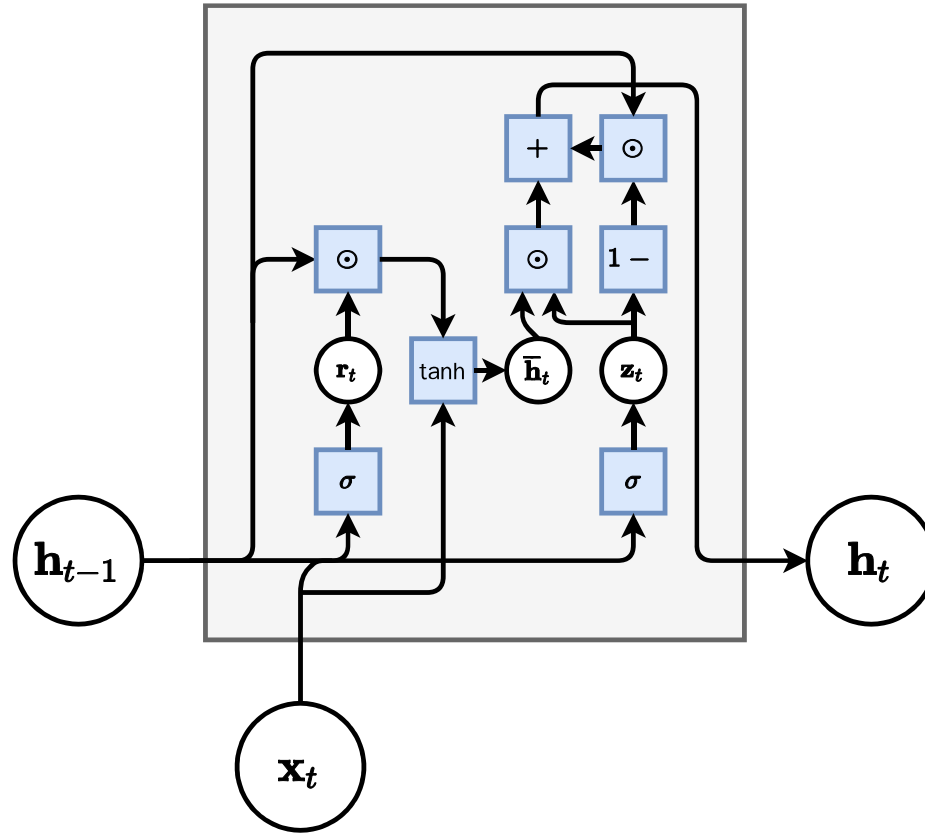
$$\mathbf{o}_t = \sigma (\mathbf{W}_o^T [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

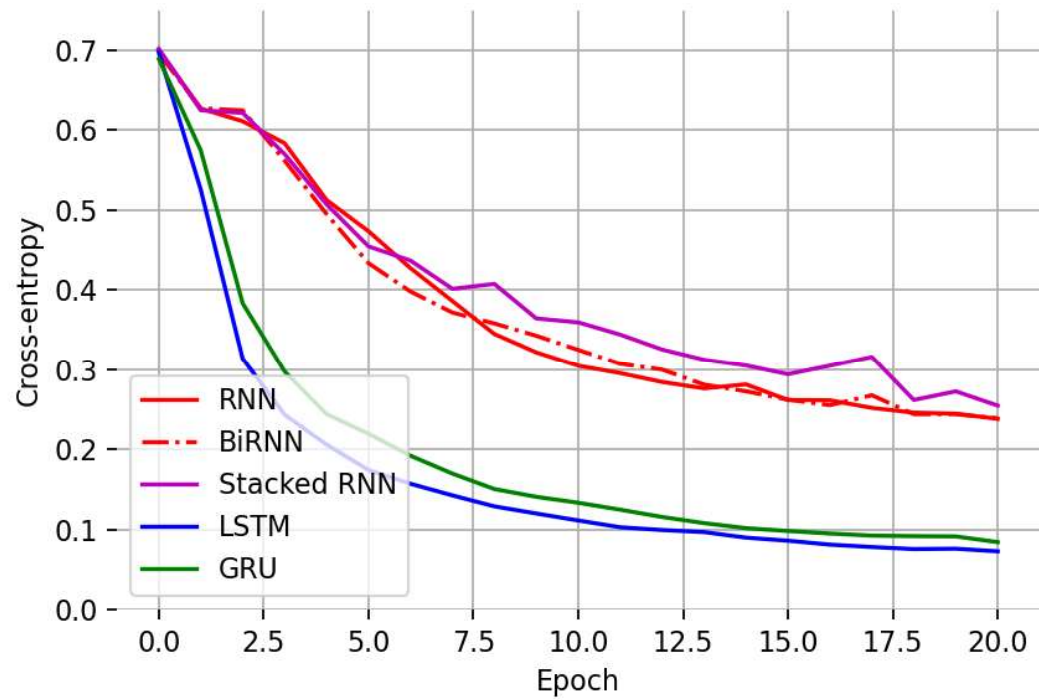


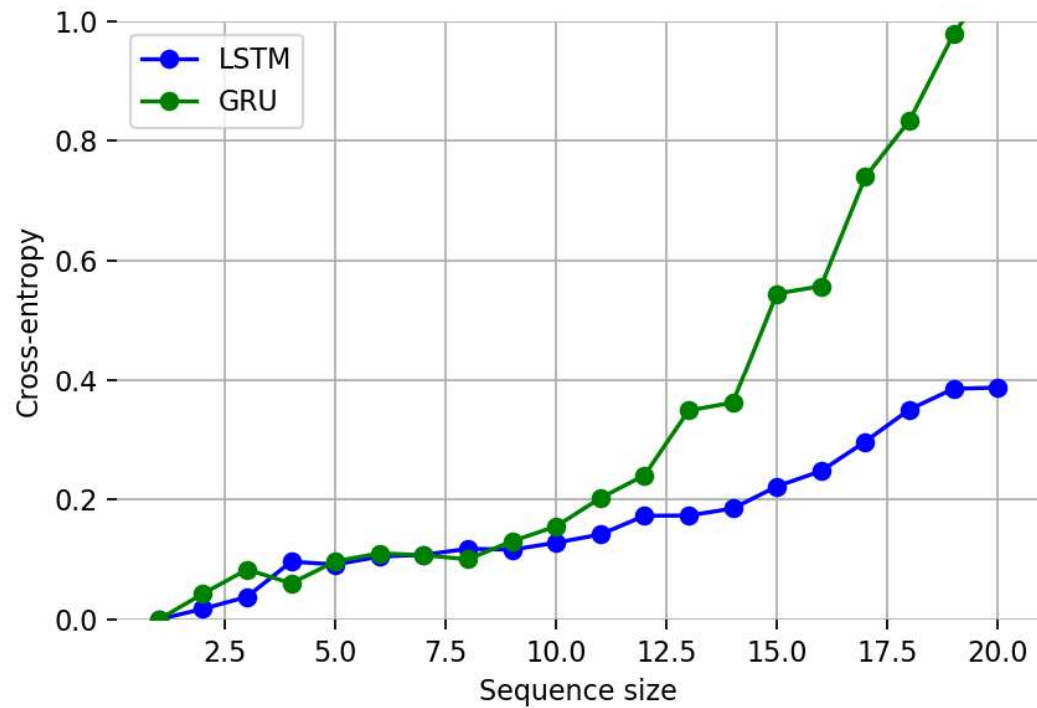
GRU

The gated recurrent unit (GRU; Cho et al, 2014) is another gated recurrent cell. It uses two gates instead of three: an update gate $\mathbf{z}_t$ and a reset gate $\mathbf{r}_t$.



$$\begin{aligned} \mathbf{z}_t &= \sigma (\mathbf{W}_z^T [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z) \\ \mathbf{r}_t &= \sigma (\mathbf{W}_r^T [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r) \\ \bar{\mathbf{h}}_t &= \tanh (\mathbf{W}_h^T [\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h) \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \bar{\mathbf{h}}_t \end{aligned}$$





The models do not generalize to sequences longer than those in the training set!

***Credits & License: Adapted from materials by Prof. Gilles Louppe (ULiège).
Source: github.com/glouppe/info8010-deep-learning Licensed under BSD 3-Clause.***